

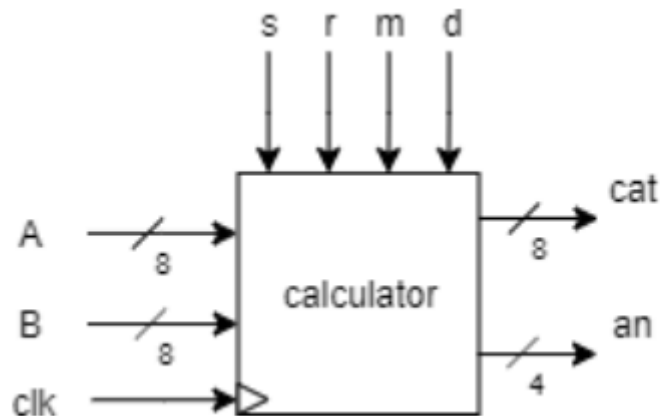
PRACTICA 4: CALCULADORA

Jordi Garcia
Mateu Magem

1. INTRODUCCIÓ

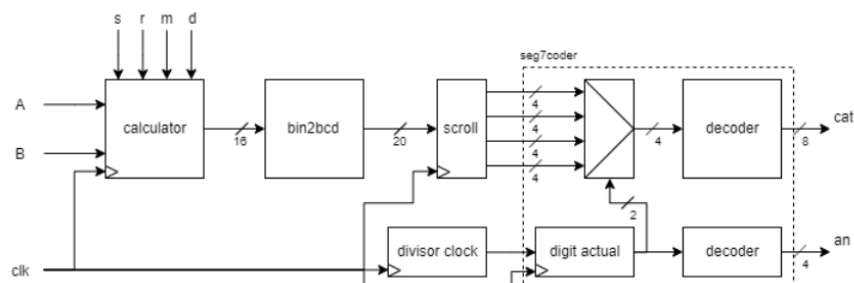
L'objectiu de la pràctica és la descripció en VHDL d'una calculadora que realitza certes operacions aritmètiques entre 2 valors de 8 bits que introduïrem mitjançant els 16 interruptors dels que disposem en la Basys 3. La operació a realitzar es seleccionarà amb 4 dels 5 polsadors de la tarja.

El polsador d'adalt (T18) ens calcularà la suma dels dos valors, el de baix (U17) la resta, el de la dreta (T17) la divisió i el de l'esquerre (W19) la multiplicació. El del mig (U18) serà el reset.



2. PROJECTES I ARQUITECTURES

A continuació descriurem els diferents components amb les seves respectives arquitectures que hem utilitzat per dur a terme la pràctica.



CALCULADORA

La calculadora és una de les parts fonamentals del projecte, a la seva entitat s'hi llegeixen els valors (A i B) entrats pels interruptors de la placa i seleccionem quina operació volem fer amb aquests.

```
entity calc is
    Port ( A,B : in STD_LOGIC_VECTOR (7 downto 0);
          s,r,m,d : in STD_LOGIC;
          clk,reset : in STD_LOGIC;
          result : out STD_LOGIC_VECTOR (15 downto 0));
end calc;
```

Com veiem, l'entitat està composta per les entrades de selecció d'operació i pels dos valors que introduïm pels interruptors. També hi tenim al clk i el reset essencials en qualsevol sistema síncron.

La sortida és el resultat de l'operació entre els valors A i B i pren una dimensió de 16 bits.

Un cop definida l'entitat toca programar el cos del projecte, l'arquitectura. La nostra arquitectura pren forma de màquina d'estats en la que relacionem cada estat amb la seva entrada i els possibles estats següents que pot prendre.

```
architecture Behavioral of calc is
    type STATE_TYPE is (E0, E1, E2, E3, E4);
    signal state, next_state : STATE_TYPE;
    signal next_result: STD_LOGIC_VECTOR(15 downto 0);
    signal aint, bint: INTEGER range 0 to 255;
    signal sm,rm,mm,dm,resm : STD_LOGIC;
```

Primer de tot definim les senyals i creem els estats necessaris del nostre projecte.

```
boto_pulsat: process (a,m,d,r,reset,sm,rm,mm,dm,resm) begin

    if s = '1' then
        sm <= '1';
        rm <= '0';
        mm <= '0';
        dm <= '0';
        resm <= '0';

    elsif m = '1' then
        sm <= '0';
        rm <= '0';
        mm <= '1';
        dm <= '0';
        resm <= '0';

    elsif r = '1' then
        sm <= '0';
        rm <= '1';
        mm <= '0';
        dm <= '0';
        resm <= '0';

    elsif d = '1' then

        sm <= '0';
        rm <= '0';
        mm <= '0';
        dm <= '1';
        resm <= '0';

    end if;
end process;
```

Primer de tot, creem un procés el qual té com a objectiu mantenir el resultat de l'operació en els displays un cop s'ha deixat de pulsar el botó. Per fer-ho creem 5 senyals que prendran el valor de 1 quan es pulsi el botó corresponent a la placa i que canviaran el seu valor a 0 quan sen pulsi un altre de diferent. D'aquesta manera farem que els botons actuïn com “d'interruptors”.

```
state_machine: process(clk, reset, resm)
begin
    if rising_edge(clk) then
        if resm = '1' then
            state <= E0;
            result <= (others => '0');
        else
            state <= next_state;
            result <= next_result;
        end if;
    end if;
end process;
```

Aquí crem el procés on es guardarà la sortida de cada estat i l'estat en el que ens trobem. Si el reset està actiu, el valor de la sortida serà 0.

```
next_state_decoder: process(state, sm, rm, mm, dm)
begin
    case (state) is
        when E0 =>
            if sm = '1' then
                next_state <= E1;
            elsif rm = '1' then
                next_state <= E2;
            elsif mm = '1' then
                next_state <= E3;
            elsif dm = '1' then
                next_state <= E4;
            else
                next_state <= E0;
            end if;
    end case;
end process;
```

Seguidament, programarem els diferents camins que ens porten d'un estat a un altre. Com veiem a la imatge ens mourem d'un estat o un altre a partir del valor de l'últim pulsador que ha estat pulsat. Si cap pulsador ha estat activat ens mantindrem a l'estat actual.

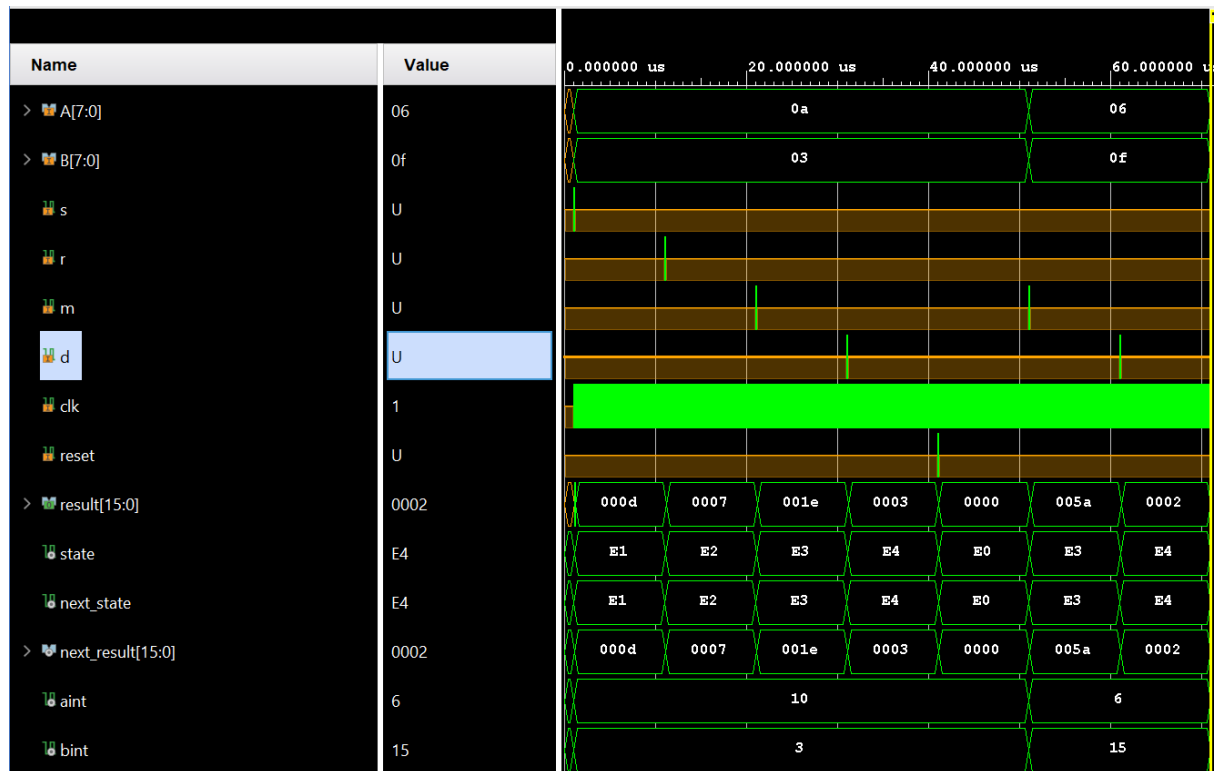
Dins d'aquest procés hi tindrem tantes seqüències *case is* com estats tingui la nostra màquina totes amb el mateix format però variant les condicions depenent de l'estat en que ens trobem.

```
output_decoder: process(state, A, B)
begin
    aint <= conv_integer(A);
    bint <= conv_integer(B);
    if state = E0 then
        next_result <= (others => '0');
    elsif state = E1 then
        next_result <= conv_std_logic_vector(aint + bint, 16);
    elsif state = E2 then
        if aint > bint then
            next_result <= conv_std_logic_vector(aint-bint, 16);
        else
            next_result <= conv_std_logic_vector(bint-aint, 16);
        end if;
    elsif state = E3 then
        next_result <= conv_std_logic_vector(bint*aint, 16);
    elsif state = E4 then
        if aint > bint then
            next_result <= conv_std_logic_vector(aint/bint, 16);
        else
            next_result <= conv_std_logic_vector(bint/aint, 16);
        end if;
    end if;
end process;
```

Finalment tindrem el procés que assignarà cada sortida al seu estat corresponent.

La forma més senzilla de realitzar-ho és passant les entrades en binari A i B a integers ja que al tractar-se d'operacions matemàtiques serà més fàcil de manipular-les.

SIMULACIÓ



Com veiem, forçant dues constants a les entrades, el clock i fent pulsos a cada entrada (pulsadors) podem veure com anem saltant d'estat a estat mentre varia el resultat (depenent de l'operació matemàtica). També podem assenyalar que al fer la divisió el nostre programa agafa el número més gran d'entre A i B per tal que el resultat sigui sempre sigui més gran de zero (com podem veure a la simulació).

SCROLL

L'scroll és la funció que ens permetrà veure el valor complet del resultat a través dels 4 displays. Això és necessari ja que el resultat de la nostra operació pot tenir fins a 5 dígits de llargada i només disposem físicament de 4 displays. L'scroll mou els valors del resultat a través dels 4 displays fins que tot el número s'ha vist completament, llavors, torna a aparèixer pel primer display.

```
entity scroll is
    generic(eoc2: integer:=50000000);
    Port ( bcd : in STD_LOGIC_VECTOR (19 downto 0);
          clk, reset : in STD_LOGIC;
          d0 : out STD_LOGIC_VECTOR (3 downto 0);
          d1 : out STD_LOGIC_VECTOR (3 downto 0);
          d2 : out STD_LOGIC_VECTOR (3 downto 0);
          d3 : out STD_LOGIC_VECTOR (3 downto 0));
end scroll;
```

L'entitat de l'scroll té com a entrada un vector de 20 bits provinent del resultat de la operació matemàtica convertida a bcd. la sortida és el valor que assignarem als displays de la placa.

```
architecture Behavioral of scroll is
    signal div_cnt : integer range 0 to eoc2;
    signal clk_div : std_logic;
    signal counter : integer range 0 to 8;
begin

    divisor_clk: process (clk, reset) begin
        if rising_edge(clk) then
            clk_div <= '0';
            if reset = '1' then
                div_cnt <= 0;
            elsif div_cnt = eoc2 then
                div_cnt <= 0;
                clk_div <= '1';
            else div_cnt <= div_cnt + 1;
            end if;
        end if;
    end process;
```

Primer de tot creem el clock que ens permetrà mostrejar els diferents valors pels diferents displays a una freqüència per veure el canvi de valor a als diferents displays (2Hz).

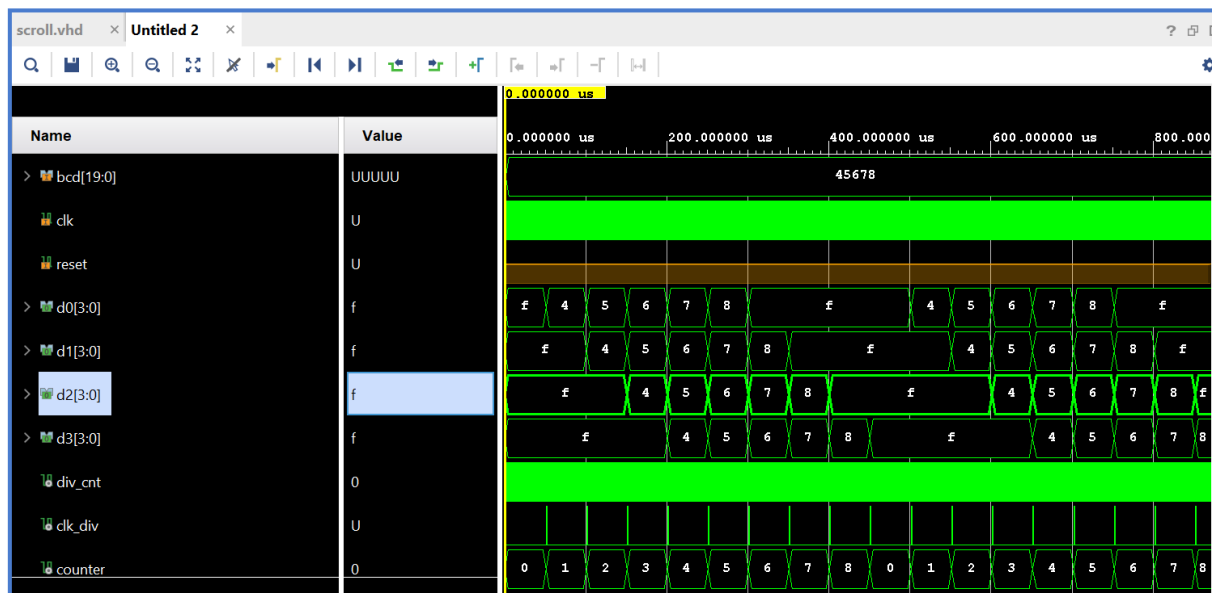
```
-- Comptador que assigna la entrada que toca
digit_counter: process(clk_div, counter, reset) begin
    if clk_div'event and clk_div = '1' then
        if reset = '1' then
            counter <= 0;
        elsif clk_div = '1' then
            if counter = 8 then
                counter <= 0;
            else counter <= counter + 1;
            end if;
        end if;
    end if;
end process;
```

Després, creem un comptador que serà el que ens faci canviar d'estat els displays a la freqüència del clock reduït.

```
scroll: process(bcd, counter)
begin
    if counter = 0 then
        d0 <= "1111";
        d1 <= "1111";
        d2 <= "1111";
        d3 <= "1111";
    elsif counter = 1 then
        d0 <= bcd(19 downto 16);
    elsif counter = 2 then
        d0 <= bcd(15 downto 12);
        d1 <= bcd(19 downto 16);
    elsif counter = 3 then
        d0 <= bcd(11 downto 8);
        d1 <= bcd(15 downto 12);
        d2 <= bcd(19 downto 16);
    elsif counter = 4 then
        d0 <= bcd(7 downto 4);
        d1 <= bcd(11 downto 8);
        d2 <= bcd(15 downto 12);
        d3 <= bcd(19 downto 16);
    elsif counter = 5 then
        d0 <= bcd(3 downto 0);
        d1 <= bcd(7 downto 4);
        d2 <= bcd(11 downto 8);
        d3 <= bcd(15 downto 12);
    end if;
end process;
```

Finalment creem el procés scroll en el que assignem un valor als displays depenent del valor en que es trobi el comptador. Com veiem, comencem donant un valor de tot uns als displays això traduït al decodificador de set segments és un guió. Com veiem a la imatge cada cop que el valor del comptador canvi anirem assignant els diferents valors de bcd als 4 displays fins que els 5 hagin completat el recorregut pels 4 displays que el procés es tornarà a repetir.

SIMULACIÓ



BIN2BCD I DEC7SEG

El binari to bcd i el decodificador a set segments son dos codis que ja venim veient durant les dues últimes pràctiques. En aquest cas, hem hagut de fer alguna petita modificació les quals mostrarem a continuació.

```
function to_bcd ( bin : std_logic_vector(n-1 downto 0) ) return std_logic_vector is
    variable i : integer:=0;
    variable bcd : std_logic_vector(19 downto 0) := (others => '0');
    variable bint : std_logic_vector(n-1 downto 0) := bin;
begin
    for i in 0 to n-1 loop -- repeating n-1 times.
        bcd(19 downto 1) := bcd(18 downto 0); --shifting the bits.
        bcd(0) := bint(n-1);
        bint(n-1 downto 1) := bint(n-2 downto 0);
        bint(0) := '0';
        if(i < n-1 and bcd(3 downto 0) > "0100") then --add 3 if BCD digit is greater than 4.
            bcd(3 downto 0) := bcd(3 downto 0) + "0011";
        end if;
        if(i < n-1 and bcd(7 downto 4) > "0100") then --add 3 if BCD digit is greater than 4.
            bcd(7 downto 4) := bcd(7 downto 4) + "0011";
        end if;
        if(i < n-1 and bcd(11 downto 8) > "0100") then --add 3 if BCD digit is greater than 4.
            bcd(11 downto 8) := bcd(11 downto 8) + "0011";
        end if;
        if(i < n-1 and bcd(15 downto 12) > "0100") then --add 3 if BCD digit is greater than 4.
            bcd(15 downto 12) := bcd(15 downto 12) + "0011";
        end if;
    end loop;
    return bcd;
end to_bcd;
```

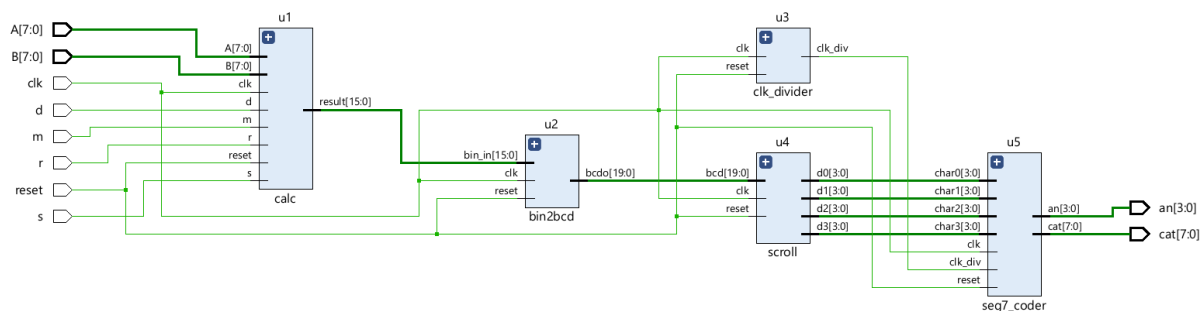
En aquest cas hem de convertir el vector de 16 bits a 5 dígit en hexadecimal i no a 4 com al cas anterior.

```
with sg select cat <=
    "00000011" when "0000", --0
    "10011111" when "0001", --1
    "00100101" when "0010", --2
    "00001101" when "0011", --3
    "10011001" when "0100", --4
    "01001001" when "0101", --5
    "01000001" when "0110", --6
    "00011111" when "0111", --7
    "00000001" when "1000", --8
    "00011001" when "1001", --9
    "11111101" when "1111"; -- -
```

Al decodificador a set segments hi hem afegit un guionet per representar l'estat inicial dels display.

SCHEMATIC

L'esquema resultant de la unió dels subprogrames és el següent:



CONCLUSIONS

Hem aconseguit implementar el subprogrames i comprovar el seu funcionament mitjançant simulacions, malgrat tot el programa en conjunt no produeix el resultat esperat, ja que produeix errors en el procés d'implementació o al ser carregat a la placa no mostra més que guions. Creiem que es tracta d'un error en el port map o de l'ús erroni del rellotje en algun punt del programa.